EchoVault Nexus

How the whole project works, end to end
A complete walkthrough script

> **What this document covers:** starting the app, every page and button, what happens behind the scenes on each click, file and screenshot attachments with text extraction (OCR), search, the AI assistant, accounts and privacy, security, the database, the API, configuration, testing, deployment and troubleshooting.
>
> **Who it is for:** anyone demonstrating, studying, maintaining or presenting the project, including complete beginners. Technical words are explained in plain language and collected in the glossary at the end.

Project folder: `~/Documents/CAAPSTOON-APF/Echovault`
Repository: github.com/Abigail56/Echovault
Document date: 29 October 2026. Reflects the code on branch `feature/visual-refresh` (pull request #7).

Contents

1. What EchoVault Nexus is
2. The problem and the solution
3. How the system is built
4. Technology used, and why
5. Starting the project
6. The running example
7. Walkthrough script, scene by scene
    - Landing page
    - Register
    - Sign in and sessions
    - Overview dashboard
    - Projects
    - Capture knowledge (notes)
    - Attach files and screenshots
    - Text extraction (OCR) and auto-insert
    - The note page and attachments panel
    - Import files as notes
    - Decisions
    - Connections map
    - Search
    - AI assistant
    - Archive, restore and delete
    - Profile and settings
    - Password reset
    - Updated terms
    - Signing out
8. How the main pages work: Vault, Projects, Decisions, Assistant
9. Life of a request (behind the scenes)
10. File upload pipeline in detail
11. Security and privacy
12. Database (data model and migrations)
13. API reference
14. Frontend structure
15. Backend structure
16. Configuration settings
17. Design system and accessibility
18. Testing and quality checks
19. Continuous integration
20. Deployment
21. Troubleshooting
22. Limitations and next steps
23. Glossary

1. What EchoVault Nexus is

EchoVault Nexus is a **private, personal knowledge workspace** that runs in a web browser. With it you can:

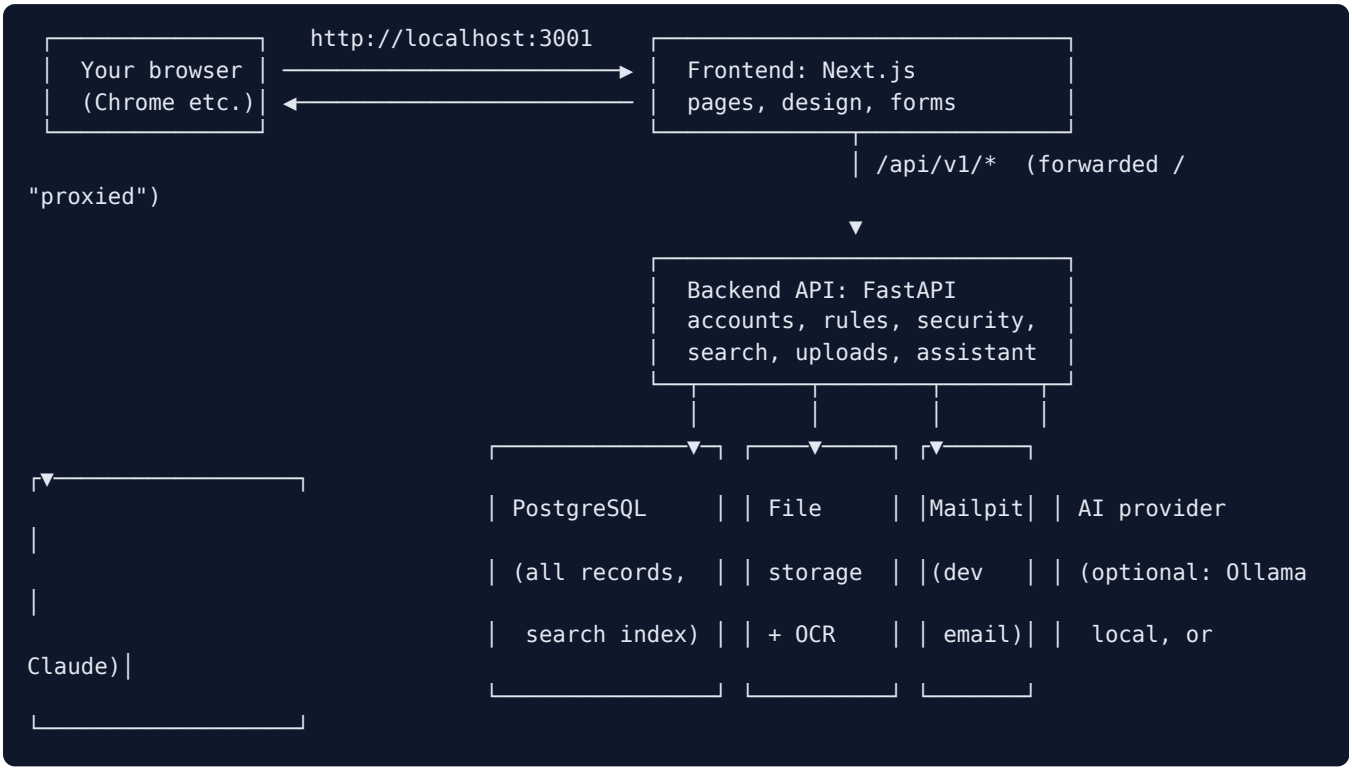
- **Capture** what you learn as notes written in Markdown, with tags.
- **Attach** files and screenshots (images, PDF, Word, Excel, PowerPoint, RTF, text) to notes. The app **reads the text inside them**, including text in screenshots, and can put that text into the note automatically.
- **Organise** notes and decisions into projects.
- **Record decisions** with the reasoning, the alternatives, and what was expected compared with what actually happened.
- **See connections** between a note, its project, that project's decisions, and related notes.
- **Search** everything at once, including the text read from attachments.
- **Ask questions** to an AI assistant that answers only from your own records and shows which records it used.

It is built for developers, students, researchers, founders, and anyone whose work depends on remembering *what they learned* and *why they decided something*.

2. The problem and the solution

Problem	What it looks like	How EchoVault Nexus solves it
Scattered knowledge	Information lives in chats, screenshots, PDFs, bookmarks and memory.	One vault for notes and their files. Text inside screenshots and documents becomes searchable.
Lost context	You remember *what* was decided but not *why*.	The Decision Journal stores context, problem, alternatives, reasoning, and expected and actual outcomes.
No connections	A note, its project and the decision it informed live in different places.	Projects, related-note links and the Connections map tie them together.
Untrustworthy AI	Chatbots answer from the internet, not from your knowledge, and you can't check their sources.	A source-grounded assistant answers only from your records and links every source. If nothing matches, it says so and sends nothing to the AI.
Privacy	Private notes get pasted into public tools.	Every record belongs to one account and the server checks ownership on every request. The AI can run fully locally with Ollama.

3. How the system is built



Part	Where it runs (development)	Job
Frontend (Next.js)	`http://localhost:3001`	Everything you see and click. Forwards every `/api/v1/…` call to the backend.
Backend API (FastAPI)	`http://127.0.0.1:8010`	Sign-in, rules, saving data, search, file uploads, text extraction, the assistant.
Database (PostgreSQL 17, Docker)	port `5433`	Stores users, sessions, notes, tags, links, projects, decisions and attachment records.
File storage	`backend/storage/attachments/`	The uploaded files themselves, saved under random names outside any public folder.
Tesseract and Poppler	installed on the computer	Free programs that read text from images (OCR) and turn scanned PDF pages into images.
Mailpit (Docker)	`http://localhost:8025`	A pretend inbox that catches emails such as password resets during development.
AI provider	Ollama at `127.0.0.1:11434` (current setup), or Anthropic Claude	Writes assistant answers from the records the backend selected.

Why the frontend forwards ("proxies") API calls: the browser only ever talks to `localhost:3001`. Next.js passes anything under `/api/v1/` to the backend. This keeps sign-in cookies *first-party* (same site), which is safer and avoids cross-site cookie problems.

4. Technology used, and why

Layer	Technology	Why it was chosen
Frontend framework	Next.js 16 (Pages Router), React 19, TypeScript (strict)	Fast pages, built-in routing, and type checking that catches mistakes before they run.
Styling	Tailwind CSS 4, DaisyUI 5, Lucide icons	A consistent design with ready-made accessible components, plus light and dark themes.
Forms	React Hook Form and Zod	Instant validation messages in the browser. The server checks everything again.
Backend	Python, FastAPI, SQLModel, Pydantic	Clear, typed API code with automatic input validation and interactive docs at `/docs`.
Database	PostgreSQL 17 (pgvector image), Alembic migrations	Reliable storage with built-in full-text search. Migrations change the tables safely, step by step.
Passwords and tokens	Argon2 (pwdlib), PyJWT	Modern password hashing and short-lived signed sign-in passes.
Files	Pillow, pypdf, Tesseract OCR, Poppler (pdftoppm), Python's zip and XML readers	Check images, read PDF text, read text from screenshots and scans, read Office files. All free.
AI	Ollama (local, free) through an OpenAI-compatible connection, or Anthropic Claude	Optional. The local option keeps notes on your computer.
Testing	pytest, Vitest, Playwright	Backend rules, frontend logic, and real browser journeys.
Automation	Docker Compose, GitHub Actions, `dev.sh`	One command to run locally, and automatic checks on every change.

5. Starting the project

5.1 One-time setup (already done on this computer)

1. Install Docker, Python 3, Node.js 22, Tesseract (`tesseract-ocr`) and Poppler (`poppler-utils`).
2. Backend: `cd backend && python3 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt`
3. Copy `backend/.env.example` to `backend/.env` and fill in the values: the database password, a long random `JWT_SECRET_KEY`, and the AI settings. **The `.env` file holds secrets. It is ignored by Git and must never be committed or shared.**
4. Frontend: `cd frontend && npm install`

5.2 Every day: one command

```
cd ~/Documents/CAAPSTOON-APF/Echovault
./dev.sh
```

What `dev.sh` does, in order:

1. Checks that the Python environment, `backend/.env` and the frontend packages exist, and that ports 8010 and 3001 are free. If a port is busy it stops with: *"Port 8010 is already in use (an old server may still be running)…"*.
2. Starts **PostgreSQL** and **Mailpit** in Docker and waits until they are healthy.
3. Runs `alembic upgrade head` so the database tables match the code.
4. Starts the **backend** (Uvicorn, auto-reload) on port 8010 and the **frontend** (Next.js dev server) on port 3001.
5. Waits until `/health` answers, then prints *"✓ EchoVault Nexus is starting. Open http://localhost:3001"*.

Keep that terminal open while you use the app. **Ctrl+C** stops both servers. The database keeps running in Docker; stop it with `cd backend && docker compose stop`. Other ports can be used, for example `BACKEND_PORT=8020 FRONTEND_PORT=3002 ./dev.sh`.

6. The running example

> **Ada** is a backend developer. Her team is launching a **public API**. She needs to remember how rate limiting works, keep a screenshot of an error from a failed deployment, record *why* the team chose a rate-limiting approach, and later ask "what rate limit did we choose and why?" without digging through old chats.

Every scene below follows Ada. Each scene has three parts: Do (what you click), See (what appears), and Behind the scenes (what the system does). Lines in *blue italics* are suggested words for a live demonstration.

7. Walkthrough script, scene by scene

Scene 1: The landing page (`/`)

DO

Open `http://localhost:3001`.

SEE

- The headline **"Your knowledge, connected to every decision."** with a blue-to-teal gradient.
- An animated **knowledge graph**: lines draw from a note to its project, a decision and references. It plays once. People who prefer reduced motion see the finished picture straight away. On phones it becomes a vertical chain.
- Feature sections, a sample decision record, and a **Privacy & security** section listing only protections that really exist.
- A footer with Terms, Privacy Policy, Acceptable Use, Cookie Policy and Contact.

BEHIND THE SCENES

This is a public page with no data from the database. Pictures are labelled *"Illustration with sample data"* so nothing pretends to be real user data.

"EchoVault keeps what you learn, the files behind it, and the decisions you make, all connected and private."

Scene 2: Creating an account (`/register`)

DO

Click **Create your vault** and fill in the form.

Field	Example	Rule
Full name	Ada Lovelace	Required, up to 120 characters.
Email	ada@example.com	Must be a valid address. Stored in lower case. Must not already be registered.
Password	rate limits are fun	10 to 128 characters. Length matters more than symbols.
Terms checkbox	ticked	Starts **unticked** and must be ticked. No marketing consent is bundled in.

BEHIND THE SCENES

1. The browser validates the form instantly (for example "Use at least 10 characters").
2. It sends `POST /api/v1/auth/register` with the details, `accept_terms: true`, and the Terms version shown on the page.
3. The server checks everything again, because a browser can never be trusted. If the Terms version is out of date it asks the user to reload and review the new Terms.
4. The database's unique rule on email stops duplicates, even if two sign-ups happen at the same moment.
5. The password is turned into an **Argon2 hash**. The real password is never stored or logged.
6. The accepted Terms version and the time are saved with the account.
7. A session is created and three cookies are set (see Scene 3). Ada lands on the **Overview**.

Rate limit: at most 10 registrations per hour from one network address.

Scene 3: Signing in and staying signed in (`/login`)

DO

Enter email and password and click **Sign in**.

SEE

The Overview, or the protected page Ada was trying to open before she signed in. A wrong password and an unknown email both show the same message: *"Invalid email or password"*. This stops attackers from learning which emails have accounts.

BEHIND THE SCENES

Cookie	What it is	Lifetime	Can page scripts read it?
`ev_access`	A short-lived signed pass (JWT) proving who you are	15 minutes	No (HttpOnly)
`ev_refresh`	A renewal token, only sent to the sign-in endpoints	14 days	No (HttpOnly)
`ev_csrf`	A random value that blocks forged requests from other websites	14 days	Yes, on purpose

1. Every request carries the access pass. The server also checks that the session has not been revoked, so signing out takes effect immediately.
2. When the pass expires, the app gets a new one with `POST /auth/refresh` and repeats the original request. The user notices nothing. The refresh token is **replaced every time**; if an old one is used again, the server assumes it was stolen and ends the session.
3. Every change (save, upload, delete) must send the CSRF value in an `X-CSRF-Token` header. Other websites cannot read it, so they cannot forge changes.

Rate limits: 8 attempts per email and 30 per network address every 15 minutes; refresh is limited to 60 per minute.

Scene 4: The Overview (`/dashboard`)

SEE

- A greeting such as **"Good morning, Ada"**.
- **Real counts** from the database: notes (with the archived count), active projects and decisions. The numbers count up briefly when the page loads.
- **Recent activity**: latest notes, projects and decisions together, newest first, each labelled with its type.
- Active projects, recently updated notes, recent decisions, and **Getting started** steps while the account is empty.
- A search bar. Pressing `/` anywhere jumps to search.

BEHIND THE SCENES

`GET /api/v1/dashboard/stats` counts only Ada's records. Nothing is hard-coded.

Scene 5: Creating a project (`/projects/new`)

DO

Go to **Projects** → **New project**. Name: *Public API launch*. Description: *Ship v1 of the public REST API with keys, docs and limits.* Status: *Active*. Click **Save**.

SEE

The project page shows the description, a **status selector** (Active, On hold, Completed, Archived), and lists of **linked notes** and **linked decisions**, each with an **Add** button that pre-selects this project.

BEHIND THE SCENES

`POST /api/v1/projects` saves the name (up to 200 characters), description (up to 5,000) and status with Ada as owner. **Deleting a project keeps its notes and decisions**; they become "not in a project". The confirmation dialog says this before anything happens.

Scene 6: Capturing knowledge (`/vault/new`)

DO

On the project page click **Add** under Knowledge (or go to **Vault** → **New item**). The page is titled **Capture knowledge**. Fill in:

- **Title:** Token bucket rate limiting
- **Content** (Markdown):

```
## How it works
Each API key gets a **bucket** that refills at a fixed rate.
- Refill: 100 requests per minute
- Burst: 20
- Over the limit: respond with HTTP 429
```

- **Project:** Public API launch (already selected)
- **Tags:** `api`, `performance` (up to 20 tags, each up to 50 characters, stored in lower case)
- **Related notes:** search "nginx" and pick *Nginx rate limit config* (up to 20 links)

SEE

- **Write / Preview** tabs. Markdown is shown safely: raw HTML is never run.
- A save status: "Unsaved changes", "Saving…" or "No unsaved changes".
- A warning if Ada tries to leave the page with unsaved changes.
- The **Attachments** area (Scene 7) between the form and the Save button.

BEHIND THE SCENES

`POST /api/v1/knowledge` stores the note (title, content, owner, project, timestamps), its tags (shared across Ada's notes), and a **two-way link** to the related note. Link targets are checked: Ada can only link to her *own* notes and projects.

Scene 7: Attaching files and screenshots while capturing

DO

Ada has a screenshot of a failed deployment saved in her Pictures folder. In the **Attachments (optional)** box she either:

- **drags** the file from her file manager onto the dotted box (it highlights and says *"Drop to add these files"*), or
- clicks **Choose Files**, which opens the computer's own file window. She can pick several files at once. Keyboard users can press Enter or Space on the button.

SEE

- Each chosen file appears in a list with a **thumbnail** (for images) or a type icon (PDF, document, spreadsheet, presentation, text), its extension and size, for example *PNG · 84 KB*.
- The status line says **"Not uploaded yet: click Save to vault"**. The file is still only in the browser.
- An **✕ Remove** button on each file.
- The hint lists accepted types and limits: *PDF (.pdf) · Documents (.doc, .docx, .txt, .md, .rtf) · Spreadsheets (.csv, .xls, .xlsx) · Presentations (.ppt, .pptx) · Images (.png, .jpg, .jpeg, .webp, .gif). Up to 10 MB per file, 20 files per note.*
- A ticked checkbox: **"Put the text from these files into the note"**.
- If a file can't be added, a red message lists each one and the reason: *"This file type isn't supported."*, *"The file is empty."*, *"Too large (12.4 MB). The limit is 10 MB."*, *"Already added."* (same name and size), or *"A note can have at most 20 files."*

BEHIND THE SCENES

These first checks run in the browser (`frontend/src/lib/attachments.ts`) for instant feedback. The limits come from `GET /api/v1/attachments/usage`, so the page always matches the server. **The server checks every file again** from its actual bytes (Section 10).

Scene 8: Saving, uploading, and extracting text automatically

DO

Click **Save to vault**.

SEE

1. Each file shows a **progress bar with a percentage**, then **"Uploaded"** with a green tick.
2. A message: *"Saved to your vault with 1 attachment"*. The note's page opens.
3. Under **Attachments** the screenshot shows *"Reading text…"*.
4. A few seconds later the text read from the screenshot appears **inside the note** under a heading such as **"From deploy-error.png"**, with the message *"Text from deploy-error.png added to the note"*.

Example: a screenshot showing *"Deploy failed at 3pm / Error: database timeout"* becomes these lines in the note. This was checked on the real running app in Chrome.

IF SOMETHING GOES WRONG

- If a file fails to upload (for example the network drops), the note is still saved. A green bar says *"Your note is saved. Open it or retry the files below."*, the failed file is marked in red with the reason, and the button changes to **Retry upload**. Retrying uploads only the failed files and **never creates a second note**. Nothing typed is lost.
- Untick the checkbox if you want the files attached without their text copied into the note. The text is still searchable either way.

BEHIND THE SCENES

1. `POST /api/v1/knowledge` creates the note. On a retry, `PATCH /knowledge/{id}` saves any edits to the same note instead.
2. Each file is sent one at a time with `POST /api/v1/knowledge/{id}/attachments`. The browser reports progress while sending; this upload also carries the CSRF header and refreshes the sign-in pass once if it expired.
3. The server saves the file and replies immediately. Reading text (OCR) can take seconds, so it runs as a **background job** after the reply.
4. The browser opens `/vault/{id}?autotext={attachment ids}`. The attachments panel checks every 2 seconds (for up to about 3 minutes) until extraction is finished.
5. When the text is ready, the page fetches the latest version of the note, adds `### From {filename}` followed by the text, saves it with `PATCH`, and removes `?autotext` from the address so a page refresh doesn't add it twice.

Scene 9: The note page (`/vault/{id}`) and the Attachments panel

SEE

- The rendered note, and a side panel with project, related notes, tags, and created/updated dates.
- Buttons: **Edit**, **Archive**, **Delete**, and **Attach files** (also available while editing).
- The **Connections** map (Scene 12).
- The **Attachments** panel with a large **"Choose files from your computer"** button and a drop area, plus storage used compared with the 200 MB quota.

Each attachment row shows an icon or image thumbnail, the file name, size, and a status badge:

Badge	Meaning
Reading text…	Extraction is waiting or running (`pending`).
Text ready	Text was read (`done`). It may be empty if the file had no words.
Couldn't read	The file could not be read (`failed`). **Retry** is offered.
OCR not installed	Tesseract is missing on the server (`unavailable`). Install it and press **Retry**.
Stored (text not read)	Old `.doc/.xls/.ppt` formats are kept but their text isn't read (`unsupported`).

Row actions:

- **View text**: opens a window with the extracted text, with **Copy** and **Add to note** buttons. Add to note re-reads the note first so newer edits are never overwritten.
- **Retry**: shown when reading failed, OCR was unavailable, or it seems stuck for more than 2 minutes. Calls `POST /attachments/{id}/extract` (20 per 10 minutes).
- **Download**: `GET /attachments/{id}/file?download=true` .
- **Delete**: asks for confirmation, then removes the database row and the file on disk.

Clicking an image thumbnail opens the image itself. Images are the only type shown inside the browser; every other type is always downloaded, for safety.

Scene 10: Importing files as notes (`/vault` → Import files)

DO

On the Vault list click **Import files** and choose one or more files.

SEE

The button shows *"Importing 1/3"*, then each file becomes **its own note**, titled from the file name (*"Q3-budget_final.pdf"* becomes *"Q3 budget final"*) and tagged `imported` . One file opens its note; several show *"Imported 3 files. Their text is being read and will be searchable shortly."*

BEHIND THE SCENES

For each file: create a note, then upload the file. If the server refuses the file, the empty note is deleted so nothing is left behind, and an error message names the file.

Scene 11: Recording a decision (`/decisions/new`)

DO

Field	Ada's entry
Title (up to 300 characters)	Rate-limit the public API per key
Status	Accepted (others: Proposed, Superseded, Reviewed)
Project	Public API launch
Context	A single client can exhaust the database connection pool.
Problem statement	Stop abusive bursts without blocking legitimate integrations.
Alternatives considered	Per-IP limits at the proxy; scale the database instead.
Selected option	Token bucket per API key, 100 requests/minute with bursts of 20.
Reasoning	Fair per customer, and needs no extra infrastructure.
Expected outcome	No more pool exhaustion; under 1% of requests limited.

SEE

The decision page puts the **selected option and reasoning first**, then context and problem, the alternatives, and **expected versus actual outcome side by side**. Statuses always show an icon and a word, never colour alone.

Six weeks later Ada clicks **Add review notes** and writes the actual outcome ("Zero pool incidents; 0.3% limited"). The **last reviewed** date is recorded automatically. **Mark reviewed** sets the status to Reviewed.

Scene 12: Seeing connections



Every box is a real record and a clickable link. The connecting lines grow into place when the page opens (instantly for reduced-motion users).

Scene 13: Search (`/search`)

DO

Press `/`, type `limit`, press Enter. Then try `timeout`.

SEE

- Results grouped into **Notes**, **Projects** and **Decisions**, with counts on filter tabs (All, Notes, Projects, Decisions) and pagination for single-type tabs.
- **Partial words match**: `limit` finds "limiting" and "limits".
- Matching words are **highlighted**. Tags match too (`perf` finds notes tagged `performance`).
- **Words inside attachments match**: `timeout` finds Ada's note because the screenshot contains "database timeout". The result shows *"In attachment: deploy-error.png"* and a preview of that text.
- Status labels such as *Accepted* or *Archived*.

BEHIND THE SCENES

`GET /api/v1/search?q=…` uses PostgreSQL full-text search with prefix matching and fast GIN indexes, over notes, tags, projects, decisions, and attachment file names and extracted text. Every query is restricted to Ada's own records. Highlighting never inserts HTML.

Scene 14: The AI assistant (`/assistant`)

DO

Ask: **"What rate limit does our public API use, and why?"**

SEE

An answer such as: *"Our public API uses a token bucket per API key: 100 requests per minute with a burst of 20 [1]. It was chosen because it is fair per customer and needs no extra infrastructure [2]."* Below it, the **sources**: each a link to the original note or decision, marked *cited* or *retrieved but not cited*.

Asking something unrelated ("What is the capital of Mongolia?") shows: *"I couldn't find anything in your notes or decisions that relates to this question, so I haven't sent it to the AI provider."*

BEHIND THE SCENES

1. **Retrieve:** search only Ada's notes and decisions for matching words (archived notes are skipped). The best matches, up to 6, are chosen. Text read from a note's attachments is included as `[Attached file: name]`.
2. **No match means no AI call.** Nothing is sent anywhere.
3. **Package safely:** each record is wrapped as a numbered `<source>` (up to 6,000 characters each) and marked as untrusted data. The instructions tell the model never to follow commands hidden inside a note.
4. **Answer:** the provider writes an answer using only those sources and cites them as [1], [2]. General knowledge, if used, must be labelled.
5. **Check:** only citation numbers that really exist are linked.

Provider	Cost	Speed on a laptop	Notes
Ollama with `qwen2.5:1.5b` (current setup)	Free	about 15–40 s	Runs on this computer; notes never leave it. A small model, so citations are less reliable.
Anthropic Claude	Paid per use	a few seconds	Best quality and citations. Needs an API key with credit, kept only in `backend/.env`.

If no provider is configured, the page says so honestly. It never shows invented answers. **Rate limit:** 20 questions per 5 minutes.

Scene 15: Archive, restore and delete

- **Archive** a note: it disappears from the main Vault list and from the assistant, but stays searchable. **Restore** it from the **Archived** tab.
- **Delete** a note, project or decision through a confirmation dialog. Deleting a note also deletes its attachment files from disk. Deleting a project keeps its notes and decisions.
- The Vault list has a search box, tag filter, project filter, sort order, Active/Archived tabs and pagination. Filters are kept in the address bar, so a filtered view can be refreshed or bookmarked.

Scene 16: Profile and Settings

Where	Action	What happens
Profile	Change name	Saved immediately (`PATCH /auth/me`).
Profile	Change sign-in email	Requires the current password. The **old** address gets a warning email that doesn't reveal the new one. 5 tries per 15 minutes.
Profile	Change password	Requires the current password; signs out every *other* device.
Settings	Theme: Dark, Light or System	Remembered in this browser.
Settings	**Download my data**	A JSON file (format version 2) with the account, notes (including archived), tags, links, projects, decisions and attachment details with their extracted text. 10 per hour.
Settings	Terms accepted	Shows the version and date accepted.
Settings	**Sign out everywhere**	Ends all sessions on all devices.
Settings	**Delete account**	Requires the password and typing `DELETE` . Permanently removes the account, all records, and all uploaded files.

Scene 17: Forgot password (`/forgot-password` , `/reset-password`)

1. On the sign-in page click **Forgot password** and enter the email.
2. The same message appears whether or not the email exists, so nobody can probe for accounts.
3. An email with a **single-use link** valid for **30 minutes** is sent. In development it appears in Mailpit at `http://localhost:8025` .
4. The link opens `/reset-password` ; Ada sets a new password. The token is stored only as a hash, and all her sessions are ended.

Limits: 5 requests per 15 minutes per address, 3 per hour per email, 10 confirmations per 15 minutes.

Scene 18: Updated Terms

If the Terms change, every page shows a prompt asking Ada to review and accept the new version (the box starts unticked) before continuing. Settings stays usable, so she can export or delete her data instead of accepting.

Scene 19: Signing out

Sign out (user menu) calls `POST /auth/logout`: the session is revoked on the server and the cookies are cleared. Going back to a protected page sends Ada to the sign-in page.

8. How the main pages work

This section explains, page by page, everything on the four pages you use most: the **Vault**, **Projects**, **Decisions** and the **Assistant**. Each is reached from the left-hand navigation (on phones, from the menu button).

8.1 The Vault page (`/vault`)

The Vault is the list of all your knowledge notes. Its job is to help you find, filter and open notes quickly.

Part of the page	What it does
Header buttons	**New item** opens *Capture knowledge* (`/vault/new`). **Import files** turns each chosen file into its own note (Scene 10).
Active / Archived tabs	"Which notes to show". Active hides archived notes; Archived shows only them; open one and click **Restore** to bring it back. Empty archive shows *"No archived notes"*.
Search box	"Search titles and content". Typing waits a moment (debounce) and then filters the list; it also matches text read from attachments.
Tag filter	"All tags" or one tag; tags are listed with how many notes use them.
Project filter	"All projects" or one project.
Sort order	**Recently updated** (default), **Newest first**, **Oldest first**, **Title A–Z**.
Note cards	Title, a two-line preview (or *"No content yet"*), up to 5 tags, project, a paperclip with the attachment count, and *"Updated 2 hours ago"*. Clicking opens the note page (Scene 9).
Pagination	20 notes per page.
Empty and no-match states	*"Your vault is empty"* with a button to capture the first note; *"No matching items. Try different keywords, or clear the filters."* with a **Clear filters** button.

Behind the scenes: every filter is written into the address bar (`?q=…&tag=…&project=…&sort=…&view=archived&offset=…`), so a filtered view survives a refresh, the Back button works, and the link can be bookmarked. The page calls `GET /api/v1/knowledge` with those values and `GET /api/v1/tags` for the tag list. Only your own notes are ever returned.

Typical use: *"I'll filter by the tag 'api', sort by newest, and open the rate-limiting note."*

8.2 The Projects pages

Projects list (`/projects`)

Part of the page	What it does
New project button	Opens `/projects/new` : name (required, up to 200 characters), description (up to 5,000) and status.
Search box	"Search names and descriptions".
Status filter	"All statuses", Active, On hold, Completed or Archived.
Project cards	Name, status badge, description, and counts such as *"3 notes · 1 decision"*. Click to open.

Pagination	24 projects per page.
Empty states	*"No projects yet"* or *"No matching projects"*.

Project page (`/projects/{id}`)

- **Header:** breadcrumb (Projects › name), status badge, and a **status selector**. Changing it saves straight away and shows *"Status updated"*.
- **Description** (or *"No description yet."*).
- **Knowledge** section: the notes linked to this project, an **Add** button that opens New item with the project already selected, and a **View all** link to the Vault filtered to this project. Empty: *"No knowledge linked to this project yet."*
- **Decisions** section: the same for decisions, with **Add** (New decision with the project selected) and **View all**. Empty: *"No decisions recorded for this project yet."*
- **Edit project** switches to an inline form; **Save changes** shows *"Project updated"*.
- **Delete** asks *"Delete this project?"* and explains that notes and decisions are kept. Afterwards: *"Project deleted. Its notes and decisions were kept."*

Behind the scenes: `GET /projects/{id}` loads the project; the linked lists use `GET /knowledge?project_id=…` and `GET /decisions?project_id=…`. On delete, the database sets `project_id` to empty on those notes and decisions instead of deleting them.

8.3 The Decisions pages (Decision Journal)

Decisions list (`/decisions`)

Part of the page	What it does
New decision button	Opens `/decisions/new` (fields in Scene 11).
Search box	"Search context, reasoning and outcomes". Searches every text field of a decision, not just the title.
Status filter	All, Proposed, Accepted, Superseded, Reviewed.
Project filter	"All projects" or one project.
Order	Newest first or Oldest first.
Decision cards	Title, status badge (icon and word), project, a short preview and the date.
Pagination and empty states	20 per page; *"No decisions recorded"* or *"No matching decisions. Try different keywords or another project."* with Clear filters.

Decision page (`/decisions/{id}`)

1. **Header:** title, status badge, project link, created date and *"Last reviewed …"* once reviewed.
2. **Selected option** and **Reasoning** first, because they are what people look up most.
3. **Context** and **Problem statement**.
4. **Alternatives considered** (or *"No alternatives recorded"*).
5. **Expected vs actual outcome** side by side. If no actual outcome yet: *"Not reviewed yet. Add review notes"*.

6. **Status panel:** a hint explaining the current status, a **Change status** selector, and a **Mark reviewed** button.
7. **Edit decision** (inline form, *"Decision updated"*) and **Delete** (*"Delete this decision?"*, then *"Decision deleted"*).

Status	When to use it
Proposed	Suggested but not agreed yet.
Accepted	Agreed and in effect (the default).
Superseded	Replaced by a later decision.
Reviewed	Looked back on, with the actual outcome recorded.

Behind the scenes: changes use `PATCH /decisions/{id}`. When the actual outcome is filled in or the status becomes Reviewed, the server records `reviewed_at` automatically. Decisions are included in search and in the assistant's sources.

8.4 The Assistant page (`/assistant`)

The Assistant is a chat page that answers questions using **only your own notes, attachment text and decisions**.

Part of the page	What it does
Status check	On opening, `GET /assistant/status` checks whether an AI provider is set up. If not, the page says so and shows which settings to put in `backend/.env` ("To enable it, set these in…"). It never fakes answers.
Suggested questions	Click to ask: *"What have I written about this project's architecture?"*, *"Summarise my notes on deployment."*, *"Which decisions mention the database?"*
Question box	"Ask a question about your notes…". **Enter** sends; **Shift+Enter** adds a new line. The **Send question** button does the same.
"Limit to a project"	"All knowledge" or one project, so only that project's notes and decisions are searched.
Conversation	Your question, then *"Searching your notes and writing an answer…"*, then the answer in Markdown. The page scrolls to the newest message.
Sources	"Sources from your notes and decisions": numbered cards [1], [2]… with an icon for note or decision, the title and an excerpt. Each links to the original record. Cited sources come first; others are marked *"Retrieved but not cited in the answer"*.
Model line	*"Generated by qwen2.5:1.5b. AI answers can be wrong; check the sources."*
No-match reply	Labelled *"System message (no AI provider was called)"*, explaining nothing relevant was found.
Errors	Timeouts, provider problems or the rate limit (20 questions per 5 minutes) appear as a red message in the conversation; you can ask again.

Behind the scenes: `POST /assistant/ask` with the question and optional project. The full step-by-step flow (retrieve, no-match check, safe packaging, answer, citation check) is in Scene 14. The conversation is kept only on the page; it is not saved to the database, and refreshing clears it.

9. Life of a request (behind the scenes)

Example: Ada clicks **Save** on a note edit.

1. **Form check** (React Hook Form + Zod) in the browser.
2. **API client** (`frontend/src/lib/api/client.ts`) sends `PATCH /api/v1/knowledge/{id}` with JSON, the cookies, and the `X-CSRF-Token` header copied from the `ev_csrf` cookie.
3. **Next.js** forwards the call to the backend at port 8010.
4. **Middleware** rejects bodies over 1 MB (10.1 MB for uploads) and adds safety headers: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Cache-Control: no-store`.
5. **Dependencies** (`app/api/deps.py`): check the CSRF header matches the cookie, decode the access JWT, and confirm the session is still active.
6. **Validation**: Pydantic checks every field (lengths, tag rules, IDs).
7. **Service** (`knowledge_service.py`): loads the note *where id matches AND owner is Ada*. Someone else's note returns "Not found", even with the right ID.
8. **Database**: changes are saved in one transaction.
9. **Reply**: the updated note as JSON. Errors return a clear message; technical details go only to the server log, which never contains passwords, tokens or note content.
10. **Expired pass?** If the reply is 401, the client calls `/auth/refresh` once and repeats the request. If refresh fails, Ada is sent to sign in, returning to the same page afterwards.

10. The file upload pipeline in detail

9.1 Receiving and checking the file

1. The note must belong to the user and have fewer than 20 attachments.
2. The file is **streamed to disk** in 1 MB pieces under a **random name** (32 hex characters), counting size as it goes. It stops as soon as the 10 MB limit or the user's remaining quota (200 MB total) is crossed. Empty files are refused.
3. A **SHA-256 fingerprint** of the content is recorded.
4. The **type is detected from the bytes**, never from the name or the browser's claim:
   - PNG, JPEG, GIF and WebP by their signatures; PDF by `%PDF-`.
   - Word, Excel and PowerPoint (`.docx/.xlsx/.pptx`) are ZIP files; the server looks inside for `[Content_Types].xml` and the `word/`, `xl/` or `ppt/` folder. **Macro-enabled files** (containing `vbaProject.bin` or a macro content type) and other ZIPs are refused.
   - Old `.doc/.xls/.ppt` share one container format, so the extension decides which they are.
   - RTF by its `{\rtf` start. Text files must be valid UTF-8 without null bytes.
   - **SVG and HTML are refused** because they can contain scripts.
5. Images are opened with Pillow to make sure they are real, undamaged images.
6. If any check fails, the partial file is deleted and a clear message is returned, such as *"This file type isn't supported. Upload images (PNG, JPEG, GIF, WebP), PDF, Word (.doc, .docx), Excel (.xls, .xlsx), PowerPoint (.ppt, .pptx), RTF, or text files (.txt, .md, .csv)"*.

7. A database row is saved with the cleaned file name, type, size, fingerprint, storage name and status `pending`. Upload limit: 60 per 10 minutes per user.

9.2 Reading the text (background job)

File type	How text is read
Images and screenshots	Rotated upright, converted to greyscale, enlarged up to 3× if narrower than 1,200 pixels, then read by **Tesseract OCR** (language `eng`, 60-second limit). Images over 50 million pixels are refused ("decompression bomb" protection).
PDF	The text layer is read with **pypdf** (up to 200 pages). If there's almost no text (a scanned PDF), up to 10 pages are turned into images with **pdftoppm** and read with OCR.
Word (.docx)	Paragraphs, tabs and line breaks from `word/document.xml`.
PowerPoint (.pptx)	Text of each slide in order, headed "Slide 1", "Slide 2"…
Excel (.xlsx)	Each sheet ("Sheet 1"…), rows as tab-separated cells, using the shared text table.
RTF	A small built-in reader keeps visible text and special characters and drops formatting and hidden groups.
Text (.txt, .md, .csv)	Read as UTF-8.
Old .doc/.xls/.ppt	Stored and downloadable; marked `unsupported` for text.

Text is tidied (extra blank lines removed) and capped at 200,000 characters. Each part of an Office file is capped at 20 MB to guard against "zip bombs". If the file or note was deleted while OCR was running, the job simply finishes without error. File contents are never written to logs.

9.3 Downloading safely

Files are served only through `GET /attachments/{id}/file` after an ownership check, with a strict `Content-Security-Policy … sandbox` header and `nosniff`. Only images display inline; everything else downloads.

11. Security and privacy

Protection	How it is implemented
Passwords	Argon2 hashing; never stored or logged in plain text. 10–128 characters.
Sessions	HttpOnly cookies, 15-minute access passes, rotating refresh tokens with reuse detection, server-side revocation, "sign out everywhere".
Forged requests (CSRF)	Double-submit token in the `X-CSRF-Token` header on every change; cookies are `SameSite=Lax`.
Account isolation	Every query filters by the signed-in user. Others' records return "Not found". IDs are random UUIDs. Tests check this across users.
Abuse	Rate limits on sign-in, registration, refresh, password reset, email and password change, export, account deletion, uploads, extraction retries and the assistant.
Uploads	Type from bytes, SVG/HTML and macros refused, image verification, size, count and quota limits, random storage names outside public folders, sandboxed downloads.
Safe content	Markdown rendered without raw HTML; search highlighting never injects HTML.
AI privacy	Only the user's own, relevant, non-archived records are sent, and only when they ask. Nothing is sent when nothing matches. Hidden instructions in notes are treated as data.
Request size and headers	Oversized bodies refused; `nosniff`, `DENY` framing, strict referrer and no-store caching on every response. CORS allows only the frontend's address.
Production mode	API docs (`/docs`) are hidden; rate limiting cannot be switched off; cookies must be `Secure` over HTTPS.
Secrets	Kept only in `backend/.env`, which Git ignores. Never printed, logged or committed.
Legal	Terms, Privacy, Acceptable Use, Cookies and Contact pages describe the real implementation; Terms acceptance is recorded with version and time.

12. Database: data model and migrations

Table	Holds
`users`	Name, email (unique, lower case), Argon2 password hash, accepted Terms version and time.
`user_sessions`	One row per signed-in device: hashed refresh token, expiry, revoked time, browser description.
`password_reset_tokens`	Hashed single-use reset tokens with expiry.
`projects`	Name, description, status (active, on_hold, completed, archived), owner.
`knowledge_items`	Notes: title, Markdown content, project, archived time, owner, timestamps.
`tags` , `knowledge_item_tags`	Per-user tags and which notes use them.

`knowledge_links`	Two-way "related note" links.
`decisions`	Title, context, problem, alternatives, selected option, reasoning, expected and actual outcome, status, reviewed time, project, owner.
`attachments`	Owner, note, file name, kind (image, pdf, text, document), detected content type, size, SHA-256, random storage key, extraction status (pending, done, failed, unavailable, unsupported), extracted text, created time.

Every table with user content has an owner. Deleting a user or a note **cascades**: their rows are removed by the database, and the app removes the matching files from disk.

Migrations (Alembic, in `backend/alembic/versions/`), applied in order:

1. `325d6ef0a69e` initial schema
2. `9d3d52ac0b6f` archive, related notes, decision status
3. `5521fecd08fc` attachments
4. `7c1e2a9b4d10` attachment documents (Office/RTF kind and `unsupported` status)

CI runs `alembic upgrade head && alembic check` to prove the migrations match the code. Downgrades are tested only on the test database, never on real data.

13. API reference

All paths start with `/api/v1`. Everything except register, login, refresh, config and password reset requires a signed-in session. Every change (POST, PATCH, DELETE) requires the CSRF header.

Method and path	Purpose
Accounts	
GET `/auth/config`	Current Terms version and whether email sending is configured.
POST `/auth/register`	Create an account and sign in.
POST `/auth/login`	Sign in.
POST `/auth/refresh`	Swap the refresh token for new cookies.
POST `/auth/logout`, `/auth/logout-all`	Sign out this device, or every device.
GET / PATCH `/auth/me`	Read or update the profile.
POST `/auth/change-email`, `/auth/change-password`	Change sign-in details (current password required).
POST `/auth/password-reset/request`, `/confirm`	Email a reset link; set a new password.
POST `/auth/accept-terms`	Accept a new Terms version.
GET `/auth/export`	Download all your data as JSON.
POST `/auth/delete-account`	Permanently delete the account.
Knowledge (notes)	
GET `/knowledge`	List with search, tag, project, archived, sort and paging.
POST `/knowledge`	Create a note.
GET / PATCH / DELETE `/knowledge/{id}`	Read, edit (including archive/restore), delete.
GET `/tags`	Your tags with usage counts.
Attachments	
POST `/knowledge/{id}/attachments`	Upload one file (form field `file`); text is read in the background.
GET `/knowledge/{id}/attachments`	List a note's attachments.
GET `/attachments/usage`	Storage used and quota, file size and count limits, whether OCR is available.
GET `/attachments/{id}`	Details including the extracted text.
GET `/attachments/{id}/file`	The file itself (`?download=true` forces download).
POST `/attachments/{id}/extract`	Read the text again.

DELETE `/attachments/{id}`	Delete the attachment and its file.
Projects and decisions	
GET / POST `/projects`; GET / PATCH / DELETE `/projects/{id}`	Project list, create, read, edit, delete.
GET / POST `/decisions`; GET / PATCH / DELETE `/decisions/{id}`	Decision list, create, read, edit and review, delete.
Search, dashboard, assistant	
GET `/search`	Search notes, projects, decisions and attachment text.
GET `/dashboard/stats`	Counts and recent items for the Overview.
GET `/assistant/status`	Whether an AI provider is configured.
POST `/assistant/ask`	Ask a question; returns the answer and sources.
Outside `/api/v1`	
GET `/health`	`{"status":"ok","database":"ok"}`, or 503 if the database is down.
GET `/docs`	Interactive API documentation (development only).

14. Frontend structure (`frontend/src`)

Folder	Contents
`pages/`	One file per address: `index` (landing), `login`, `register`, `forgot-password`, `reset-password`, `dashboard`, `vault/index`, `vault/new`, `vault/[id]`, `projects/…`, `decisions/…`, `search`, `assistant`, `profile`, `settings`, the legal pages (`terms`, `privacy`, `acceptable-use`, `cookies`, `contact`) and `404`.
`components/attachments/`	`AttachmentPicker` (drag-and-drop staging on New item), `AttachmentPanel` (list, status, retry, auto-insert on the note page), `ExtractionBadge`, `TextViewer`, `ImportFilesButton`.
`components/knowledge, projects, decisions`	Forms, cards and pickers for each record type.
`components/layout`	`AppShell` (side navigation, user menu, sign-in guard), `PublicLayout`, `TermsUpdateGate`.
`components/ui`	Shared pieces: Logo, Markdown, ConnectionMap, ConfirmDialog, SearchInput, Pagination, TagInput, SaveStatus, States (loading, empty, error).
`hooks/`	`useAuth`, `useToast`, `useTheme`, `useAsync`, `useUnsavedChanges`, `useUrlFilters`, `useDebounce`, `useCountUp`.
`lib/`	`api/client.ts` (requests, CSRF, refresh-and-retry, uploads with progress), `api/services.ts`, `api/types.ts`, `attachments.ts` (file rules), `schemas.ts` (Zod forms), `format.ts`, `legal.ts`.
`styles/globals.css`	Theme colours, shadows and effects for light and dark modes.

15. Backend structure (`backend/app`)

Folder	Contents
`main.py`	Creates the app: CORS, size limit and safety headers, error handlers, `/health`.
`api/v1/endpoints/`	Thin route files: `auth`, `knowledge`, `attachments`, `projects`, `decisions`, `search`, `assistant`. `router.py` adds the CSRF check to protected routes.
`api/deps.py`	Shared checks: database session, signed-in user, CSRF.
`services/`	The business rules: `auth_service`, `knowledge_service`, `project_service`, `decision_service`, `search_service`, `search_vectors`, `assistant_service`, `attachment_service`, `file_types`, `extraction`, `export_service`, `email_service`.
`models/`	Database tables (SQLModel).
`schemas/`	The shapes of requests and replies, with validation rules.
`core/`	`config` (settings), `security` (hashing, JWT), `cookies`, `rate_limit`, `legal` (Terms version).
`ai/providers.py`	Connections to Anthropic and to OpenAI-compatible servers such as Ollama.
`db/session.py`	The database connection.

16. Configuration settings (`backend/.env`)

Setting	Default	Meaning
`ENVIRONMENT`	development	development, test or production.
`DATABASE_URL`	local PostgreSQL on 5433	Where the database is.
`JWT_SECRET_KEY`	(required, at least 32 characters)	Signs sign-in passes. Secret.
`ACCESS_TOKEN_EXPIRE_MINUTES`	15	Access pass lifetime.
`REFRESH_TOKEN_EXPIRE_DAYS`	14	Session lifetime.
`PASSWORD_RESET_EXPIRE_MINUTES`	30	Reset link lifetime.
`FRONTEND_ORIGIN`	http://localhost:3000 (dev.sh sets 3001)	Allowed browser origin and base for email links.
`COOKIE_SECURE`, `COOKIE_SAMESITE`, `COOKIE_DOMAIN`	false, lax, none	Cookie settings. Secure must be true with HTTPS.
`TRUST_PROXY_HEADERS`	false	Trust `X-Forwarded-For` only behind a proxy you control.
`RATE_LIMIT_ENABLED`	true	Only switched off for browser tests; refused in production.
`MAX_REQUEST_BYTES`	1,000,000	Largest normal request body.
`ATTACHMENTS_DIR`	`backend/storage/attachments`	Where uploaded files are stored.
`MAX_UPLOAD_BYTES`	10,000,000	Largest file (10 MB).
`USER_STORAGE_QUOTA_BYTES`	200,000,000	Storage per user (200 MB).
`MAX_ATTACHMENTS_PER_NOTE`	20	Files per note.
`OCR_LANGUAGES`, `OCR_TIMEOUT_SECONDS`, `PDF_OCR_MAX_PAGES`	eng, 60, 10	OCR language(s), time limit, scanned-PDF pages read.
`AI_PROVIDER`	(empty = assistant off)	`anthropic` or `openai_compatible` (Ollama). Current setup: openai_compatible.
`AI_API_KEY`, `AI_MODEL`, `AI_BASE_URL`	(empty)	Key (secret), model name, server address. Current: `qwen2.5:1.5b` at `http://127.0.0.1:11434/v1`.
`AI_TIMEOUT_SECONDS`, `AI_MAX_SOURCES`	45, 6	AI time limit; records sent per question.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `SMTP_FROM`, `SMTP_STARTTLS`	(empty), 587…	Email sending. In development this points at Mailpit.

The frontend reads `BACKEND_URL` (where to forward API calls); `dev.sh` sets it automatically.

17. Design system and accessibility

- **Themes:** Dark (deep navy `#070b16` background, blue `#5b8cff` primary, teal `#2dd4bf` accent) and Light (`#f4f6fb` background, `#2557e8` primary, `#0f766e` accent). Colour pairs were checked against WCAG AA contrast.
- Soft shadows, a gentle background glow, gradient headline text and a subtle shine on primary buttons.
- **Motion:** every animation respects the "reduce motion" system setting.
- **Accessibility:** real labels on every input; keyboard access to everything, including the file chooser (Enter/Space); visible focus rings; status shown with words and icons, never colour alone; alerts announced to screen readers; touch targets at least 44 px.
- **Responsive:** works from 320 px phones to wide screens with no sideways scrolling, which is checked by automated tests.

18. Testing and quality checks

Check	Size	What it covers
Backend tests (pytest)	100 tests, all passing at the last run	Sign-up rules, sessions, CSRF, rate limits, CRUD for every record type, archive, related links, decision status, search (including attachment text), export, email change, account deletion, assistant behaviour, provider handling, cross-user isolation, and 31 attachment tests: file-type detection, refusing SVG/HTML/macros, size, quota and count limits, OCR, PDF, Word, Excel, PowerPoint and RTF reading, retry, delete, safe download headers.
Frontend unit tests (Vitest)	32 tests, all passing at the last run	API client (CSRF, refresh, errors), sign-in form, Terms prompt, email links, highlighting, file rules (types, size, duplicates).
Browser tests (Playwright, Chromium)	33 tests in 9 files	Real journeys: register and sign in, password reset through a real email, vault, projects and decisions, search and privacy, account settings, attachments (upload, OCR, view text, delete, import), New item uploads (pick, drag-drop, remove, validation, save, automatic text into the note, failed upload then retry without a duplicate note, 360 px phone layout), and no sideways scrolling on many pages and screen widths.
Type check and lint	whole frontend	`npm run typecheck` and `npm run lint`, both clean.

How to run them:

```
cd backend  && .venv/bin/pytest -q          # needs TEST_DATABASE_URL (echovault_test)
cd frontend && npm run lint && npm run typecheck && npm test
cd frontend && npm run test:e2e             # starts its own servers on 8011 / 3011
```

> Browser tests use a separate database (`echovault_e2e`), separate ports (8011 and 3011) and a temporary folder for uploaded files, so they never touch real data. On a slow development machine the first page compile can be slow; one import test once timed out for that reason and passed when re-run. CI runs on a clean machine.

19. Continuous integration (GitHub Actions)

Every push to `main` and every pull request runs three jobs (`.github/workflows/ci.yml`):

1. **Backend tests:** starts PostgreSQL, installs Tesseract and Poppler, installs Python packages, checks migrations match the models, runs pytest.
2. **Frontend checks:** lint, type check, unit tests, production build.
3. **End-to-end tests:** PostgreSQL and Mailpit services, installs Chromium, runs Playwright. On failure it uploads the report, screenshots and traces for 7 days.

Work is done on branches and merged through pull requests. PR #6 (attachments with OCR) is merged. PR #7 (visual refresh, New item uploads, Office documents, automatic text insertion) is open on `feature/visual-refresh` .

20. Deployment

The app has not yet been deployed to the internet. The pieces are ready:

- `backend/Dockerfile` builds the API with Tesseract and Poppler installed and a `/app/storage` volume for uploaded files.
- Production needs: `ENVIRONMENT=production`, HTTPS with `COOKIE_SECURE=true`, a strong `JWT_SECRET_KEY`, a managed PostgreSQL, a real SMTP service, `FRONTEND_ORIGIN` set to the real site, persistent storage for attachments, backups for both the database and the files, and `alembic upgrade head` on each release.
- The frontend is built with `npm run build` and started with `BACKEND_URL` pointing at the API.
- The rate limiter keeps counts in memory, so a single API instance is assumed; several instances would need a shared store such as Redis.

21. Troubleshooting

Problem	Cause	Fix
"Port 8010 is already in use" when running `./dev.sh`	An earlier `dev.sh` is still running in another terminal.	Press Ctrl+C in that terminal, or find it with `ss -ltnp | grep 8010` and stop it. Or use other ports: `BACKEND_PORT=8020 FRONTEND_PORT=3002 ./dev.sh`.
`localhost:3001` won't open	`dev.sh` isn't running, or the first compile is still in progress.	Run `./dev.sh`, keep the terminal open, wait for the ✓ line, then reload.
Design changes don't appear, or the page looks old	The Next.js dev cache is stale, or the computer ran out of file watchers.	Stop `dev.sh`, run `rm -rf frontend/.next/dev`, start again, then press Ctrl+Shift+R. If it keeps happening, raise `fs.inotify.max_user_instances` (now set to 1024 in `/etc/sysctl.d/99-inotify.conf`).
"Choose Files" doesn't open a window	The browser's file dialog is blocked; seen once when Chrome had been running for days with errors.	Fully close and reopen Chrome. Meanwhile, drag the file from the file manager onto the dotted box, which always works.
Screenshot attached but no text appears	The note was never saved: chosen files show "Not uploaded yet". Or Tesseract isn't installed.	Click **Save to vault**. If the badge says "OCR not installed", install it (`sudo apt install tesseract-ocr poppler-utils`) and click **Retry**.
Text stuck on "Reading text…"	The backend was restarted during OCR.	After 2 minutes a **Retry** button appears; click it.
Assistant says no provider is configured, or times out	`AI_PROVIDER` is empty, or Ollama isn't running.	Start Ollama (`ollama serve`, model `qwen2.5:1.5b`) or configure Claude in `backend/.env`, then restart `dev.sh`.
Password reset email not received	In development emails go to Mailpit, not a real inbox.	Open `http://localhost:8025`.
"Too many attempts. Please wait and try again."	A rate limit was reached.	Wait for the time shown; limits reset automatically.
Backend says the database is unavailable	Docker isn't running.	Start Docker, then `./dev.sh` again. Check with `curl http://127.0.0.1:8010/health`.

22. Limitations and next steps

- Search is keyword-based (full-text), not meaning-based. Semantic search with pgvector (already in the database image) is a planned improvement.
- OCR reads English by default and works best on clear screenshots; handwriting and low-quality photos give weaker results. More languages can be added via `OCR_LANGUAGES`.
- Text isn't read from old `.doc/.xls/.ppt` files (they are stored and downloadable). Scanned PDFs are OCR'd up to 10 pages.

- Text extraction runs inside the API process; heavy use would call for a separate job queue.
- The free local AI model is slower and less precise than Claude; Claude needs a funded API key.
- A new email address isn't confirmed by a link; the change takes effect after the password check.
- Rate-limit counters live in memory (one server instance).
- Legal pages contain [placeholders] for business details and need a lawyer's review before a public launch.
- The app has not been deployed to the internet yet.

23. Glossary

Term	Meaning
API	The set of web addresses the frontend calls to read and change data.
Argon2	A modern, deliberately slow password-hashing method that makes stolen password lists hard to crack.
Background job	Work done after the reply is sent, so the user doesn't wait (here: reading text from files).
CSRF	Cross-Site Request Forgery: a trick where another website makes your browser act on a site you're signed into.
Full-text search	Searching the words inside documents, with word stems and ranking.
HttpOnly cookie	A cookie page scripts cannot read, which protects sign-in tokens from theft.
JWT	A signed token that proves identity for a short time.
Markdown	Simple text formatting: `**bold**`, `## heading`, `- list`.
Migration	A versioned script that changes the database structure safely.
OCR	Optical Character Recognition: reading the words in a picture, such as a screenshot.
Proxy	A middle-man that forwards requests; here Next.js forwards API calls to the backend.
Rate limit	A cap on how many times an action can be done in a period, to stop abuse.
SHA-256	A fingerprint of a file's contents.
Source-grounded AI	An AI answer built only from supplied documents, with references back to them.
Tesseract	A free OCR program used to read text from images and scanned PDFs.
UUID	A long random ID that can't be guessed.

EchoVault Nexus: built with Next.js, FastAPI and PostgreSQL. For setup commands and deployment details, see the project README.